

04_m7_m9

M7 — Minimal Autonomous Loop (L1 EXIT)

Objective

Stand up the agent framework (A8.0) and the smallest end-to-end run: a deterministic Python FSM that goes `PLAN → MONITOR → NARRATE` , invokes the **Monitor** (Sonnet) and **Narrator** (Sonnet, `render_brief` only) agents as model-driven nodes, and emits a Decision Brief. The framework is the single enforcement point — it owns the per-agent tool allow-list, the `dry_run` -before-`run_query` gate (G3), the byte budget, and the audit log. This is the L1 exit: **one injected anomaly produces a brief in under 5 minutes with zero hallucinated columns** (every SQL string in the audit log originated from `semantic.build_query`). The LLM never controls flow and never authors SQL (AGENT_ARCHITECTURE §1, §13).

Files to create

Path	Copy/spec from
<code>helios/agents/config.py</code>	AGENT_ARCHITECTURE §9 (the tunables: <code>MAX_REFUTE_ROUNDS=2</code> , <code>MAX_BRANCHING=4</code> , <code>MAX_DEPTH=3</code> , <code>MIN_RATE_EFFECT=0.3pp</code> , <code>MIN_SEGMENT_SESSIONS=500</code> , <code>SIGNIFICANCE_ALPHA=0.05</code> , <code>ANOMALY_SCORE_THRESHOLD=3.0</code> , <code>PRIOR_HALF_LIFE_DAYS=60</code> , <code>BYTE_BUDGET=5 GiB</code> , <code>TIME_BUDGET_MIN=5</code> , <code>MODELS</code> map, <code>TEMPERATURE=0.0</code> , <code>RNG_SEED=1729</code>)
<code>helios/agents/framework.py</code>	AGENT_ARCHITECTURE §4 — <code>AgentSpec</code> dataclass (§4.1), the tool-call wrapper (§4.2: allow-list assert → <code>run_query</code> ⇒ <code>dry_run_seen</code> assert → byte-budget enforce → retries → <code>append_audit_log</code>), the <code>Finding</code> schema + validator (§4.3), context compaction (§4.4)
<code>helios/agents/monitor.py</code>	AGENT_ARCHITECTURE §6.2 — <code>MONITOR_SPEC</code> (tools: <code>semantic.get_metric</code> , <code>semantic.build_query</code> , <code>warehouse.dry_run</code> , <code>warehouse.run_query</code> , <code>stats.detect_anomaly</code> , <code>stats.forecast</code>) + system prompt sketch

Path	Copy/spec from
helios/agents/narrator.py	AGENT_ARCHITECTURE §6.7 — NARRATOR_SPEC ; M7 variant: render_brief only (no save_diagnosis until M8). Tools at M7: report.render_brief, semantic.get_metric
helios/runner.py	AGENT_ARCHITECTURE §5 (FSM diagram + §5.1 transition table) + §10 (run trace). M7 path = the trivial PLAN→MONITOR→NARRATE subset
helios/agents/tests/fixtures/	Recorded MCP tool fixtures (canned build_query / run_query / detect_anomaly responses) per AGENT_ARCHITECTURE §12

Note: models/semantic/semantic_layer.yaml, eval/scenarios/*.yaml, and docs/CLAUDE.md **already exist — do not regenerate**. The MCP servers (helios/mcp/*.py) come from M6/M6b and must be running/importable before M7.

Commands to run

```
# Windows: .venv\Scripts\activate | POSIX: source .venv/bin/activate
source .venv/bin/activate
export ANTHROPIC_API_KEY=... # Windows: $env:ANTHROPIC_API_KEY="..."
export GOOGLE_APPLICATION_CREDENTIALS=./helios-wh-sa.json
export HELIOS_WH_TOKEN=...

# 1) framework + node unit tests with recorded fixtures (no live BigQuery)
pytest helios/agents/tests -q

# 2) one real minimal run over a window with a known dip (clean-run also valid)
python -m helios.runner --t0 2020-12-01 --t1 2020-12-20 --window-t1b 2021-01-10 \
    --metrics session_conversion_rate --budget-gib 5 --time-budget-min 5

# 3) prove grounding: every sql_text in the audit log came from semantic.build_query
python -m helios.runner --audit-check --run-id <last_run_id>
```

Tests to run

```
pytest helios/agents/tests/test_framework.py -q # allow-list + dry_run gate + audit
pytest helios/agents/tests/test_monitor.py -q # returns anomaly list envelope from fixtures
pytest helios/agents/tests/test_fsm.py -q # PLAN→MONITOR→NARRATE + clean-run short-circuit
```

- test_framework.py : asserts a tool **outside** the spec raises AllowListViolation ; calling run_query without a prior dry_run on the same normalized SQL raises NotDryRunFirst ; every call appends exactly one audit_log row.

- `test_monitor.py` : feeding the recorded daily-series fixture yields an anomaly-list envelope `[{metric, dim, t0, t1, score, direction}]` validated against `output_schema`.
- `test_fsm.py` : a zero-anomaly Monitor envelope routes straight to `NARRATE` (no-finding brief) → `END`; a one-anomaly envelope still completes (Decompose/Diagnose are stubs at M7 — route `MONITOR`→`NARRATE` directly).

Success criteria

- `pytest helios/agents/tests` is all-green.
- A single injected/real anomaly produces a rendered Decision Brief in **< 5 min** wall-clock (`run_state.ended_at - started_at`).
- `--audit-check` reports **0 hand-authored SQL**: every `audit_log.sql_text` is non-NULL only for `semantic-mcp.build_query`-sourced queries and **0 hallucinated columns/metrics** (every metric name resolved by `semantic-mcp`).
- The clean-run short-circuit works: zero anomalies still writes a `run_state` row and a "no material change" brief (absence of finding is recorded).

Common mistakes

- **Making the LLM the controller.** The runner is plain Python; the agent only composes tool calls and returns a typed `Finding[]` /anomaly envelope. If the model decides transitions, you have lost auditability and determinism (AGENT_ARCHITECTURE §1).
- **Allow-list as advice, not enforcement.** The §4.2 wrapper must `assert tool in agent.allowed_tools` for every call. A prompt that "asks nicely" is not enforcement — structural beats behavioral (§7).
- **Skipping the dry_run gate.** Calling `run_query` without a prior `dry_run` on the same SQL must raise `NotDryRunFirst` (G3). Do not retry an over-budget `dry_run` blindly — narrow window/dims first (§11).
- **Dumping row-level results into the LLM context.** Reduce `run_query` output to aggregates / `(slice, rate, p, n)` tuples before it enters the model window (§4.4); otherwise token cost explodes as the tree widens.
- **Expecting byte-identical LLM output.** Run at `temperature 0` but grade *statistically* via the eval harness (M10), not by string-equality. Seed only the stats (`RNG_SEED=1729`); the audit log makes the run reconstructable.
- **Building M7 before M6/M6b are green.** Monitor needs live `semantic.build_query`, `warehouse.dry_run/run_query`, and `stats.detect_anomaly`. Wire them first.

M8 — Memory Store (L2)

Objective

Make Helios *learn across runs*. Create the eight `helios_memory` BigQuery tables (the durable system of record), a vector store for embedding-based similarity recall, seed the calendars/glossary, and implement the `report-mcp` memory tools (`save_diagnosis` / `recall_prior` , A7.4) so agents read/write memory **only** through `report-mcp` (the LLM never writes memory directly). This is the prerequisite for the full Critic refutation battery (it needs the seasonality/launch priors) and for the Narrator's `save_diagnosis` . Exit: `save_diagnosis` → `recall_prior` round-trips, and the seasonality calendar is seeded with the dataset window's known events (Bible §22).

Files to create

Path	Copy/spec from
<code>sql/helios_memory_ddl.sql</code>	Bible §22.1–§22.5 — the 8 tables: <code>diagnosis_history</code> , <code>suppression_list</code> , <code>glossary</code> , <code>seasonality_calendar</code> , <code>launch_calendar</code> , <code>action_tracking</code> , <code>run_state</code> , <code>audit_log</code> (copy the <code>CREATE TABLE IF NOT EXISTS</code> statements verbatim, including <code>PARTITION BY</code> / <code>CLUSTER BY</code>)
<code>seeds/memory/seasonality_calendar.csv</code>	Bible §22.3 — seed Black Friday 2020 (start 2020-11-27, end 2020-11-30, <code>expected_dir=up</code>), December peak ("Christmas dip"/holiday, ~2020-12-01→2020-12-25), New Year / January trough (2020-12-26→2021-01-31, <code>expected_dir=down</code>) with <code>expected_mag</code> for confound subtraction
<code>seeds/memory/launch_calendar.csv</code>	Bible §22.3 — known GA4-sample launches, e.g. <code>landing_page=/sale</code> (<code>affected_dims</code> , <code>affected_metric</code>)
<code>seeds/memory/glossary.csv</code>	Bible §22.3 + CLAUDE.md §4 canonical names — synonym → <code>canonical_name</code> map (e.g. "checkout drop-off" → <code>checkout_to_purchase_rate</code>)
<code>scripts/init_memory.py</code>	Bible §22 — applies the DDL, loads the three seed CSVs, computes glossary/seasonality embeddings, initializes the vector store collection
<code>helios/mcp/report.py</code> (memory tools)	MCP_ARCHITECTURE §6.5 + §9. Add <code>save_diagnosis(diagnosis)</code> and <code>recall_prior(metric, segment)</code> to the existing <code>report-mcp</code> from M6b

Path	Copy/spec from
mcp_servers.yaml (update)	MCP_ARCHITECTURE §3 — set report-mcp's helios_memory dataset + vector_store: \${HELIOS_VECTOR_STORE_URI}

models/semantic/semantic_layer.yaml, eval/scenarios/*.yaml, CLAUDE.md **already exist — do not regenerate.**

Commands to run

```
source .venv/bin/activate
export HELIOS_VECTOR_STORE_URI=... # e.g. local FAISS path or managed ANN endpoint

# 1) create the 8 tables in helios_memory
bq query --use_legacy_sql=false --location=US < sql/helios_memory_ddl.sql

# 2) seed calendars + glossary and build the vector collection
python scripts/init_memory.py --seeds seeds/memory --location US

# 3) start report-mcp with memory tools live (stdio)
python -m helios.mcp.report

# 4) round-trip smoke test
python -m helios.mcp.tests.memory_roundtrip # save then recall the same finding
```

Tests to run

```
pytest helios/mcp/tests/test_report_memory.py -q
```

- Asserts `save_diagnosis(finding)` inserts one `diagnosis_history` row **and** upserts `(finding_id, embedding, metric, root_cause_label)` into the vector store; re-saving the same `finding_id` **upserts, not duplicates** (idempotent on `finding_id`, MCP_ARCHITECTURE §8).
- Asserts `recall_prior(metric, segment)` returns the just-saved prior via the hybrid query (exact filter on `metric` + `dimension_slice` U vector ANN), with recency weight `exp(-age_days/60)` applied.
- Asserts `seasonality_calendar` contains the BF2020 / December / January rows after seeding (a `SELECT COUNT(*)` ≥ 3 with the expected `event_label`s).
- Asserts `save_diagnosis` with a malformed envelope raises `ValidationError`.

Success criteria

- All 8 tables exist in `helios_memory` (`bq ls helios_memory` shows them with the §22 partition/cluster specs).

- `save_diagnosis` → `recall_prior` round-trips: a saved finding is retrievable by (`metric`, `segment`) and by vector similarity.
- `seasonality_calendar` is seeded with Black Friday 2020, the December peak, and the January trough (the priors the Critic needs at M9).
- `report-mcp` is the **only** memory writer — no agent and no other server holds a `helios_memory` write path.

Common mistakes

- **Not seeding the seasonality calendar.** Without BF2020/Dec/Jan rows, the M9 Critic is blind to known seasonality and will PASS expected moves as findings (Bible §22.3). Seed these first.
- **Suppression rows with no TTL.** `suppression_list.expires_at` defaults to a 30-day TTL so a re-emerging issue eventually re-surfaces; a permanent `NULL` should be deliberate, not the default — otherwise issues never come back.
- **Letting the LLM write memory directly.** All memory I/O goes through `report-mcp`; do not give any agent a raw `helios_memory` insert path (Bible §22, AGENT_ARCHITECTURE §8).
- **Giving `report-mcp` a BigQuery *query* client for analysis.** It reads/writes `helios_memory` only; it is **not** an analytics path and must never run governed marts SQL (least-privilege, MCP_ARCHITECTURE §9). Keep its scope to memory tables + vector store.
- **Non-idempotent `save_diagnosis`.** Key the upsert on `finding_id`; re-running a window must upsert, not double-write (MCP_ARCHITECTURE §8).
- **Hard-deleting stale priors.** Decay with `exp(-age_days/60)`; never delete (audit requirement, Bible §22.1). `expires_at` reads filter the suppression list; they don't drop history rows.
- **Wrong dataset location.** `helios_memory` must be created with `--location=US` to join the US-resident marts; a mismatched location yields "dataset not found".

M9 — Full 7-Agent Loop (L2 core)

Objective

Complete the deterministic FSM and the five remaining agents so a run produces **Critic-approved** findings, each carrying a decomposition, a significance test, a dollar revenue-at-risk, and a prescribed powered experiment. Add **Decompose** (Sonnet), **Diagnose** (Opus, hypothesis-tree RCA), **Critic** (Opus, adversarial refutation loop), **Prescribe** (Sonnet), and **Orchestrator** (Opus, drives the FSM + budget + routing). Extend `runner.py` to the full

`PLAN→MONITOR→DECOMPOSE→DIAGNOSE→[CRITIC loop]→PRESCRIBE→NARRATE→END` machine with the §5.1 transitions, wire `A9.3` audit on every transition, and upgrade the Narrator to call `save_diagnosis` (now that M8 memory exists). Exit: the run yields PASS findings with significance +

dollar impact + an experiment card, and the FSM routes correctly (clean-run short-circuit; bounded DOWNGRADE re-query; DROP→suppression) (AGENT_ARCHITECTURE §5–§10).

Files to create

Path	Copy/spec from
<code>helios/agents/decompose.py</code>	AGENT_ARCHITECTURE §6.3 — <code>DECOMPOSE_SPEC</code> ; per anomaly <code>build_query</code> the <code>[{seg,w,r}]</code> table at <code>t0/t1</code> , call <code>stats.decompose_change</code> → <code>{mix,rate,interaction,by_segment}</code> , label <code>dominant</code> . Tools: <code>semantic.build_query</code> , <code>warehouse.dry_run</code> , <code>warehouse.run_query</code> , <code>stats.decompose_change</code>
<code>helios/agents/diagnose.py</code>	AGENT_ARCHITECTURE §6.4 — <code>DIAGNOSE_SPEC</code> + the best-first hypothesis-tree (expand by rate-effect magnitude; prune on <code>MIN_RATE_EFFECT</code> / <code>SIGNIFICANCE_ALPHA</code> / <code>MIN_SEGMENT_SESSIONS</code> ; bounds <code>MAX_BRANCHING</code> / <code>MAX_DEPTH</code> ; leaf promotion requires significant + single-dim + <code>reconcile</code> ≤0.5% + dollar-material; <code>QUANTIFY</code> <code>revenue_at_risk</code> = (<code>conv_t0</code> − <code>conv_t1</code>)× <code>sessions_t1</code> × <code>aov</code>)
<code>helios/agents/critic.py</code>	AGENT_ARCHITECTURE §6.5 — <code>CRITIC_SPEC</code> + the 4-axis refutation battery (mix-shift confound, insufficient sample, seasonality via <code>recall_prior</code> + <code>seasonality_calendar</code> , data-quality probe) → verdict PASS/DOWNGRADE/DROP. Tools: <code>semantic.build_query</code> , <code>warehouse.dry_run/run_query/reconcile</code> , <code>stats.significance_test</code> , <code>stats.decompose_change</code> , <code>report.recall_prior</code>
<code>helios/agents/prescribe.py</code>	AGENT_ARCHITECTURE §6.6 — <code>PRESCRIBE_SPEC</code> ; <code>power_analysis</code> → <code>runtime_estimate</code> → <code>design_experiment</code> , rank by ICE, write <code>action_tracking(proposed)</code> . Tools: <code>experiment.power_analysis</code> , <code>experiment.runtime_estimate</code> , <code>experiment.design_experiment</code> , <code>semantic.get_metric</code>
<code>helios/agents/orchestrator.py</code>	AGENT_ARCHITECTURE §6.1 — <code>ORCHESTRATOR_SPEC</code> ; open <code>run_state(PLAN)</code> , <code>recall_prior</code> priors+suppression, set scope/budget, drive FSM, route every candidate to Critic, finalize/ABORT. Tools: <code>warehouse.list_tables/describe_table</code> , <code>semantic.list_dimensions</code> , <code>report.recall_prior</code>
<code>helios/runner.py</code> (extend)	AGENT_ARCHITECTURE §5 (full FSM + §5.1 table) + §10 trace — add <code>DECOMPOSE/DIAGNOSE/CRITIC</code> -loop/ <code>PRESCRIBE</code> states; Critic loop

Path	Copy/spec from
	bounded by MAX_REFUTE_ROUNDS
helios/agents/narrator.py (upgrade)	AGENT_ARCHITECTURE §6.7 — add report.save_diagnosis + report.export to the M7 Narrator (faithfulness rule: every number traces to a tool-output hash)
helios/agents/audit.py (A9.3)	AGENT_ARCHITECTURE §4.2 + Bible §22.5 — append_audit_log(run_id, step_seq, agent, mcp_tool, args_hash, sql_text, bytes_scanned, latency_ms, verdict, ts) wired into the framework wrapper

semantic_layer.yaml, eval/scenarios/*.yaml, CLAUDE.md already exist — do not regenerate.

Commands to run

```
source .venv/bin/activate

# 1) node + FSM + Critic unit tests with recorded fixtures
pytest helios/agents/tests -q

# 2) a full real run (Orchestrator-driven) over a window with a known dip
python -m helios.runner --full --t0 2020-12-01 --t1 2020-12-20 \
    --window-t1b 2021-01-10 --metrics session_conversion_rate \
    --budget-gib 5 --time-budget-min 5

# 3) verify routing + persistence
python -m helios.runner --audit-check --run-id <last_run_id> # 0 hand-authored SQL
bq query --use_legacy_sql=false --location=US \
    'SELECT state, n_findings, n_passed, n_dropped FROM helios_memory.run_state ORDER BY started_at
    DESC LIMIT 1'
```

Tests to run

```
pytest helios/agents/tests/test_diagnose.py -q # tree expands by rate-effect, prunes correctly
pytest helios/agents/tests/test_critic.py -q # PASS/DOWNGRADE/DROP on labeled fixtures
pytest helios/agents/tests/test_fsm_full.py -q # all §5.1 transitions
```

- test_critic.py : a pure mix-shift finding mislabeled as a rate change → DROP or DOWNGRADE ; a finding overlapping a seeded seasonality_calendar entry within expected_mag → DROP / DOWNGRADE as expected; a clean significant single-dimension finding → PASS (AGENT_ARCHITECTURE §6.5, §12).
- test_fsm_full.py : clean run (0 anomalies) short-circuits MONITOR→NARRATE→END; DOWNGRADE routes back to DIAGNOSE and is bounded by MAX_REFUTE_ROUNDS=2 (after the cap → watchlist note, not a confident finding); DROP adds a run-local suppression so siblings

don't re-raise; a hard failure/budget-exhaustion → `ABORTED` with an audit row and **no partial brief**.

- `test_diagnose.py` : drills the largest **rate-effect** slice first (not mix); prunes branches below `MIN_RATE_EFFECT`, above `SIGNIFICANCE_ALPHA`, or under `MIN_SEGMENT_SESSIONS`; promotes a leaf only if reconciled ($\leq 0.5\%$ drift) and dollar-material.

Success criteria

- A real run yields ≥ 1 **PASS Finding**, each carrying `decomposition` (mix/rate/interaction), `significance` (test + p + CI), `dollar_impact.revenue_at_risk_usd`, and a `recommendation.experiment_card_id` — the "every finding is actionable" rule (CLAUDE.md §3).
- FSM routing verified: clean-run short-circuits; `DOWNGRADE` re-queries are bounded by `MAX_REFUTE_ROUNDS`; `DROP` writes run-local suppression; failure → `ABORTED` with no partial brief.
- The audit log proves **0 hand-authored SQL** and every brief number maps to a tool-output hash (faithfulness, §12).
- `save_diagnosis` persists each PASS (and DROP, for audit) to `diagnosis_history`; Prescribe writes `action_tracking(proposed)`.
- `run_state` row closes at `END` with populated `n_findings/n_passed/n_dropped` and `brief_uri`.

(Quantitative accuracy — root-cause top-1 $\geq 85\%$ vs $\leq 45\%$ baseline, decomposition MAPE $\leq 10\%$, hallucination = 0 — is graded by the eval harness in **M10**, not asserted here.)

Common mistakes

- **Skipping the Critic / shipping unverified findings.** Every candidate must survive the §6.5 refutation battery before it reaches Prescribe/Narrator. A finding that bypasses the Critic is a defect.
- **The Critic confirming instead of refuting.** Its prompt and battery must *attack* the finding (default to skepticism); PASS only when all four refutations fail. A "looks plausible" Critic is useless.
- **Unbounded DOWNGRADE loops.** Cap Critic→Diagnose re-query at `MAX_REFUTE_ROUNDS=2`; after the cap, demote to a watchlist note, do not loop forever (§9, §5.1).
- **Drilling mix before rate.** Diagnose must order expansion by *rate-effect* magnitude — chasing mix effects first re-introduces Simpson's-paradox artifacts the decomposition exists to dissolve (§6.4, CLAUDE.md §4).
- **Promoting a leaf without reconcile.** Leaf promotion requires a `reconcile` $\leq 0.5\%$ drift (G4) and a non-trivial dollar impact; skipping reconcile ships numbers that don't tie to canonical totals.

- **LLM authoring SQL or computing stats.** No agent holds a raw-SQL tool; all SQL is `semantic.build_query`, all math is `stats-mcp / experiment-mcp` (G1/G2). The framework allow-list makes raw SQL structurally impossible — keep it that way.
- **Dumping the hypothesis tree's row-level results into context.** Summarize intermediate nodes to `(slice, rate_effect, p, n)` tuples (§4.4); the tree widens fast and unsummarized rows blow the token/cost budget.
- **Not enforcing per-agent allow-lists in the full loop.** Each of the five new agents gets exactly its §2.1 row — e.g. the Narrator still cannot call `run_query`, Prescribe cannot touch `warehouse`. Enforce structurally, not by prompt.